Defining cell types by their molecular features

Previously, we compared and contrasted **electrophysiology features**, and **structure** of different cell types in the brain. **Gene expression patterns** are also commonly used to define neuronal cell types. Since at any moment each cell makes mRNA from only a fraction of the genes it carries, by measuring the cell's RNA transcripts we can get a sense of the cell's gene expression patterns. The sum of a cell's RNA transcripts is called its **transcriptome**, and single-cell RNA sequencing (**scRNA-seq**) is one commonly used transcriptomic technology.

Here is a list of scRNA-seq data published by the Allen Institute for Brain Science:

- Adult mouse cortical cell taxonomy revealed by single cell transcriptomics (Tasic et al. 2016)
- Shared and distinct transcriptomic cell types across neocortical areas (Tasic et al. 2018)
- A taxonomy of transcriptomic cell types across the isocortex and hippocampal formation (Yao et al. 2021)

Sequencing data from scRN-seq experiments are usually contained in a matrix of expression values. This matrix is known as a **count matrix**, which contains the number of reads mapped to each gene (row) in each cell (column).

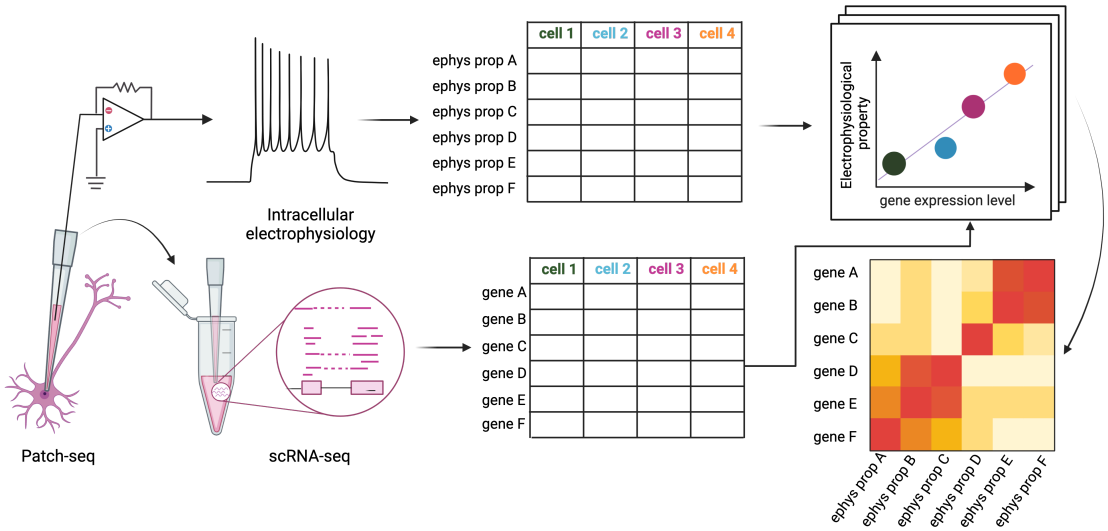
What is a 'read'?

In a typical scRNA-Seq experiment, the mRNA molecules are first reverse transcribed into cDNA molecules, which are then amplified and converted into libraries of cDNA fragments. These cDNA libraries are then sequenced, and each sequenced fragment (or read) corresponds to a portion of a cDNA molecule. We can use computer algorithms to align these reads to a reference genome or transcriptome and determine the gene(s) from which each read originates. By counting the number of reads that map to each gene, we can estimate the expression level of individual genes in individual cells.

Patch-seq

Patch-seq is an experimental technique that combined scRNA-seq with intracellular electrophysiology, which enables one to directly relate the transcriptomic features of a

given neuron to other identifying characteristics of the same neuron.



Objectives

- Obtaining normalized count matrix from raw read counts
- Exploring Patch-seq dataset generated from mouse primary visual cortex.
- Correlating gene expression with intracellular electrophsiological features.

Tutorial steps

1. Setting up our coding environment
2. Obtaining count matrix for the Patch-seq dataset
3. Normalizing count matrix
4. Plotting expression levels of ion channel genes
5. Assigning subclass labels
6. Plotting electrophysiological features across subclass
7. Correlating an electrophysiological feature with the expression of a gene
8. Constructing Gene-ephys Correlation Matrix

Step 1. Setting up our coding environment

Importing packages

In [1]:
```
# Install and then restart the kernel!
#!pip install seaborn==0.12.2
```

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import urllib.request
import tarfile
import io
import json
import umap
import gc # no longer used
import scipy.sparse as sparse
%whos
```

```
2024-04-24 18:19:22.353344: I tensorflow/core/platform/cpu_feature_guard.cc:
182] This TensorFlow binary is optimized to use available CPU instructions i
n performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
Variable   Type      Data/Info
------------------------------
gc         module    <module 'gc' (built-in)>
io         module    <module 'io' from '/opt/c<...>nda/lib/python3.9/io.py'>
json       module    <module 'json' from '/opt<...>hon3.9/json/__init__.py'>
np         module    <module 'numpy' from '/op<...>kages/numpy/__init__.py'>
pd         module    <module 'pandas' from '/h<...>ages/pandas/__init__.py'>
plt        module    <module 'matplotlib.pyplo<...>es/matplotlib/pyplot.py'>
sns        module    <module 'seaborn' from '/<...>ges/seaborn/__init__.py'>
sparse     module    <module 'scipy.sparse' fr<...>cipy/sparse/__init__.py'>
tarfile    module    <module 'tarfile' from '/<...>ib/python3.9/tarfile.py'>
umap       module    <module 'umap' from '/opt<...>ckages/umap/__init__.py'>
urllib     module    <module 'urllib' from '/o<...>n3.9/urllib/__init__.py'>
```

Step 2. Obtaining count matrix for the Patch-seq dataset

Download count matrix for the Patch-seq dataset from NeMO

Run the code cell below to download the count matrix from The Neuroscience Multi-omic Archive (NeMO). This step should take around 2 minutes, depending on your internet connection.

In [3]:
```
!wget "https://data.nemoarchive.org/other/AIBS/AIBS_patchseq/transcriptome/s
!tar -xvf 20200513_Mouse_PatchSeq_Release_count.v2.csv.tar
```

```
--2024-04-24 18:19:25--  https://data.nemoarchive.org/other/AIBS/AIBS_patchseq/transcriptome/scell/SMARTseq/processed/analysis/20200611/20200513_Mouse_PatchSeq_Release_count.v2.csv.tar
Resolving data.nemoarchive.org (data.nemoarchive.org)... 134.192.156.26
Connecting to data.nemoarchive.org (data.nemoarchive.org)|134.192.156.26|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 444999680 (424M) [application/x-tar]
Saving to: ‘20200513_Mouse_PatchSeq_Release_count.v2.csv.tar.4’

20200513_Mouse_Patc 100%[===================>] 424.38M  1.58MB/s    in 5m 31s

2024-04-24 18:24:56 (1.28 MB/s) - ‘20200513_Mouse_PatchSeq_Release_count.v2.csv.tar.4’ saved [444999680/444999680]

20200513_Mouse_PatchSeq_Release_count.v2/20200513_Mouse_PatchSeq_Release_count.v2.csv
```

Since the count matrix was deposited on NeMO as a tar file, we need to extract the csv file from the tar file using the `tarfile` package available as a part of the Python Standard Library, and then read the csv file into a Pandas DataFrame object.

Note: The cell below can only be run once. It will throw an error if you try to run it again! If you need to run it again for some reason, please restart the kernel.

In [4]:
```
count_matrix = pd.read_csv('20200513_Mouse_PatchSeq_Release_count.v2/2020051
print('count_matrix CSV file loaded as a Pandas DataFrame.')
```

```
count_matrix CSV file loaded as a Pandas DataFrame.
```

Below, we'll do some cleaning up of the `count_matrix`, and then show it.

In [5]:
```
# Set the index to be the cell ID, and rename it
count_matrix.set_index("Unnamed: 0", inplace = True)
count_matrix.index.rename('gene_name',inplace = True)

# remove genes that are not expressed in any cells
count_matrix = count_matrix.iloc[np.flatnonzero(count_matrix.sum(axis = 1) !

print(f"There are {count_matrix.shape[1]} cells and {count_matrix.shape[0]}

# Show the first 5 rows of the count matrix
count_matrix.head()
```

```
There are 4435 cells and 41930 genes in the count matrix:
```

Out[5]:

gene_name	PS0810_E1-50_S88	PS0817_E1-50_S19	PS0817_E1-50_S25	PS0817_E1-50_S26	PS0817_E1-50_S27	PSC
0610005C13Rik	0	0	0	0	0	
0610006L08Rik	0	0	0	0	0	
0610007P14Rik	0	0	0	81	51	
0610009B22Rik	0	0	41	0	0	
0610009E02Rik	0	0	0	0	0	

5 rows × 4435 columns

In this count matrix, each row represents one gene and the row names are the gene names, e.g., **0610005C13Rik**.

▶ Click here for explanation why this gene has a name that does not seem to make any sense

On the other hand, each column in the count matrix represents one cell and the column names are `transcriptomic_sample_id`, e.g., `PS0810_E1-50_S88`, which uniquely identify the transcriptomic data collected from each cell.

In [6]:
```
# This cell shows an example of how you could write the count_matrix to a cs
# However, the count_matrix is so big that this really doesn't save much tim
#count_matrix.to_csv('count_matrix.csv')
#count_matrix = pd.read_csv('count_matrix.csv')
```

Step 3. Normalizing gene counts CPM

Below, we'll import the library size for each cell (more on this later, but for now, we'll get this computationally-intensive step out of the way).

In [7]:
```
# Import precomputed library sizes for each cell, each row contains the libr
original_lib_size = pd.read_csv(urllib.request.urlopen("https://raw.githubus

# Remove cells that contain NA values
lib_size = original_lib_size.dropna().loc[original_lib_size.library_size !=

# Remove the cells that have been removed in the previous step from lib_size
count_matrix = count_matrix.loc[:, count_matrix.columns.isin(lib_size.index)

print(f"{len(original_lib_size) - len(lib_size)} cells were dropped.")

# Plot a heatmap, similar to the website! Note: this takes a long time
# count_matrix.style.background_gradient(cmap ='RdBu_r')
```

```
81 cells were dropped.
```

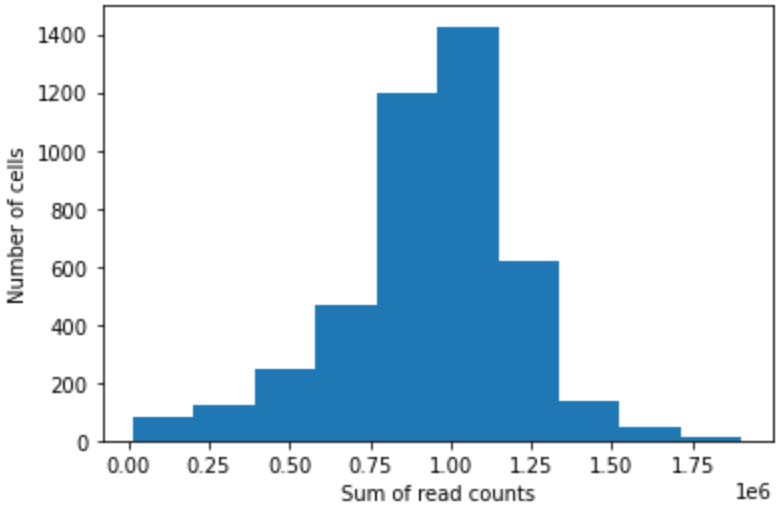
The `count_matrix` DataFrame contains raw read counts.

> **Task**: Create a histogram below showing the **sum** of raw read counts detected in individual cells. You can use the sum method (`sum()`) on your `count_matrix` dataframe, specifying `axis = 0` to sum all reads per cell. Set the number of bins to 20 for a bit more resolution on the histogram.

In [8]:
```
# Try sum
count_matrix.sum(axis=0)
```

Out[8]:
```
PS0810_E1-50_S88       985775
PS0817_E1-50_S19       252106
PS0817_E1-50_S25       588583
PS0817_E1-50_S26       532056
PS0817_E1-50_S27       308507
                         ...
SM-J39ZH_S576_E1-50    539547
SM-J3A1L_S592_E1-50    389359
SM-J3A1L_S593_E1-50    414767
SM-J3A1L_S603_E1-50     48364
SM-J3A1L_S604_E1-50    482411
Length: 4354, dtype: int64
```

In [9]:
```
# Add your plotting line below
sum_reads = count_matrix.sum(axis=0)
plt.hist(sum_reads)
plt.xlabel("Sum of read counts")
plt.ylabel("Number of cells")
plt.show()
```



As we can see, there is a wide distribution (notice 1e6 on x axis!), which is largely due to the variability inherent in the experimental technologies used to obtain the read counts, rather than biological variability. Thus, to compare gene expression levels between cells,

we need to **normalize** the raw read counts for differences in total number of reads per cells.

One common way to normalize read counts is to calculate **counts per million (CPM)**, which is obtained by dividing read counts for genes by the sum of raw read counts in that cell, also known as **library size**, and multiplying the results by a million, respectively. We imported those values above, and now we can compute the CPM.

Calculating cpm

Task: Calculate counts per million for `count_matrix`, using precomputed library sizes provided in `lib_size`.

- Note that `lib_size` is a DataFrame where the index stores the `transcriptomic_sample_id`, and the library size of the corresponding cell is stored in the `library_size` column
- To convert raw read counts for all genes expressed in a cell, we need to divide the raw read counts for each gene by the library size of the cell (which is the total number of reads) and then multiply this number by a factor of 100000.
- Below is an example of how to calculate counts per million for one cell. Your task is to calculate CPM for all cells in `count_matrix`

For the cell whose `transcriptomic_sample_id` is `PS0817_E1-50_S19`, we can extract its `library_size` using the following code:

In [10]:
```
example_lib_size = lib_size.loc['PS0817_E1-50_S19', 'library_size']
print(f"The library size for cell PS0817_E1-50_S19 is {example_lib_size}.")
```

```
The library size for cell PS0817_E1-50_S19 is 692522.0.
```

The raw read counts of all genes expressed in a cell whose `transcriptomic_sample_id` is `PS0817_E1-50_S19` would be

In [11]:
```
count_matrix["PS0817_E1-50_S19"]
```

Out[11]:
```
gene_name
0610005C13Rik     0
0610006L08Rik     0
0610007P14Rik     0
0610009B22Rik     0
0610009E02Rik     0
                 ..
Zzef1             0
Zzz3             60
a                 0
l7Rn6             2
n-R5s136          0
Name: PS0817_E1-50_S19, Length: 41930, dtype: int64
```

To calculate the CPM of a gene in a cell, we need to divide the raw read count of all genes expressed in this cell by the library size of this cell, and then multiply the result by a factor of 1000000

In [12]:
```
count_matrix['PS0817_E1-50_S19'] / lib_size.loc['PS0817_E1-50_S19', 'library
```

Out[12]:
```
gene_name
0610005C13Rik     0.000000
0610006L08Rik     0.000000
0610007P14Rik     0.000000
0610009B22Rik     0.000000
0610009E02Rik     0.000000
                   ...
Zzef1             0.000000
Zzz3             86.639847
a                 0.000000
l7Rn6             2.887995
n-R5s136          0.000000
Name: PS0817_E1-50_S19, Length: 41930, dtype: float64
```

Task (continued): Using a `for` loop, can you compute the CPM for all cells and assign the result to the corresponding entry in the `count_matrix`?

Hint 1: `count_matrix.columns` returns a list of all `transcriptomic_sample_id`, you can iterate over this list to compute the CPM for each cell

Hint 2: use the example for one cell above as a reference.

In [13]:
```
# First, we need to create an empty list to store the CPM for each cell

cpm = []

# Then, for each cell, we compute the CPM and append it to the list

for column in count_matrix.columns:

    this_cpm = count_matrix[column] / lib_size.loc[column, 'library_size'] *

    cpm.append(this_cpm)

# Finally, we concatenate the list of CPM into a dataframe (Hint: use pd.cor

cpm = pd.concat(cpm,axis=1)
```

Below, we'll inspect the normalized count matrix `cpm`:

In [14]:
```
cpm.head()
```

Out[14]:

gene_name	PS0810_E1-50_S88	PS0817_E1-50_S19	PS0817_E1-50_S25	PS0817_E1-50_S26	PS0817_E1-50_S27	PSC
0610005C13Rik	0.0	0.0	0.000000	0.000000	0.000000	0
0610006L08Rik	0.0	0.0	0.000000	0.000000	0.000000	0
0610007P14Rik	0.0	0.0	0.000000	57.737297	66.848074	23
0610009B22Rik	0.0	0.0	26.496435	0.000000	0.000000	0
0610009E02Rik	0.0	0.0	0.000000	0.000000	0.000000	0

5 rows × 4354 columns

Transpose `cpm` so that each row is one cell and each column is one gene.

In [15]:
```
cpm = cpm.T
cpm.head()
```

Out[15]:

gene_name	0610005C13Rik	0610006L08Rik	0610007P14Rik	0610009B22Rik	0610
PS0810_E1-50_S88	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S19	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S25	0.0	0.0	0.000000	26.496435	
PS0817_E1-50_S26	0.0	0.0	57.737297	0.000000	
PS0817_E1-50_S27	0.0	0.0	66.848074	0.000000	

5 rows × 41930 columns

The normalized read counts are often log-transformed for visualization purposes

In [16]:
```
cpm = np.log2(cpm + 1)
```

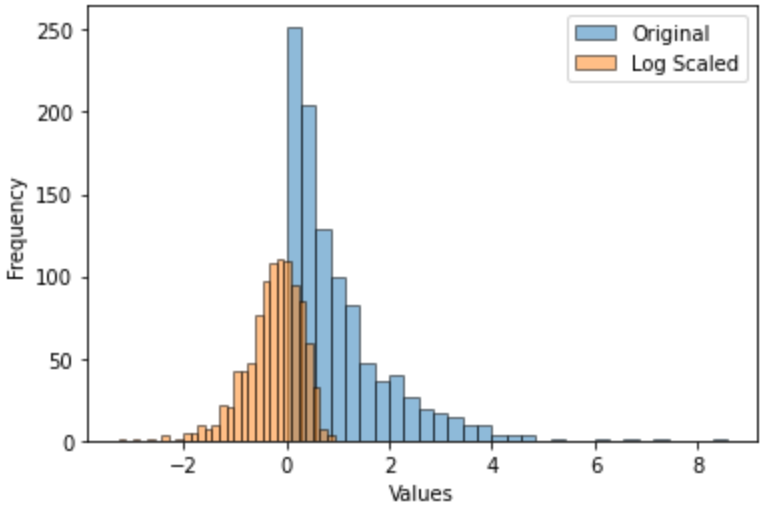
Side Note: What does "log scaling" do to our data?

In [17]:
```
# Generate skewed data
np.random.seed(0)
data = np.random.exponential(scale=1, size=1000)
data_log = np.log10(data) # change to log2 if you'd like!
```

```
print(data[:10])
print(data_log[:10])
```

```
[0.79587451 1.25593076 0.92322315 0.78720115 0.55104849 1.03815929
 0.5755192  2.22352441 3.31491218 0.4836021 ]
[-0.09915541  0.0989657  -0.03469332 -0.10391428 -0.25881018  0.016264
 -0.23994018  0.3470419   0.52047203 -0.31551182]
```

In [18]:
```
plt.hist(data, bins=30, edgecolor='black',alpha=0.5,label='Original')
plt.hist(data_log, bins=30, edgecolor='black',alpha=0.5,label='Log Scaled')
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.legend()
plt.show()
```



In [19]:
```
cpm.head()
```

Out[19]:

gene_name	0610005C13Rik	0610006L08Rik	0610007P14Rik	0610009B22Rik	0610
PS0810_E1-50_S88	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S19	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S25	0.0	0.0	0.000000	4.781173	
PS0817_E1-50_S26	0.0	0.0	5.876205	0.000000	
PS0817_E1-50_S27	0.0	0.0	6.084236	0.000000	

5 rows × 41930 columns

Step 4. Plotting expression levels of ion channel genes

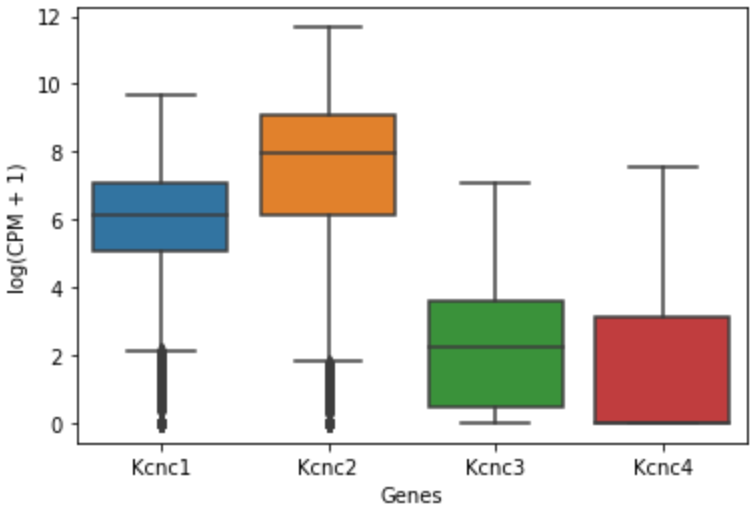
Now that we have normalized and log-transformed the raw read counts, we can present the expression levels of selected genes in all cells as a boxplot. The code below creates a boxplot of expression levels of Kv3 voltage-gated potassium channels (Kcnc1, Kcnc2, Kcnc3 and Kcnc4). These channels are known to contribute to neurons' ability to generate high-frequency activity.

In [20]:
```
# You can put any genes in this list
gene_list = ["Kcnc1", "Kcnc2", "Kcnc3", "Kcnc4"]

# Reshape the DataFrame using melt (only necessary for sns==0.11.1)
#melted_cpm = pd.melt(cpm[gene_list], var_name="Gene")
```

In [21]:
```
# Plot the melted DataFrame
#sns.boxplot(x="Gene", y="value", data=melted_cpm) #for sns==0.11
sns.boxplot(cpm[gene_list]) # simpler syntax, only works with sns==0.12.1

plt.xlabel('Genes')
plt.ylabel('log(CPM + 1)')
plt.show()
```



However, plotting the expression levels of genes across all cells in the dataset like this is not very helpful. What we want to know is whether the expression level of a gene is different between different neuronal subclasses. Since this dataset contains subclass label for each neuron, we can try to address this question by plotting gene expression across different subclasses.

Step 5. Assigning subclass labels

Obtaining subclass labels

Since each neuron has already been assigned a subclass label in the paper, our task is to obtain subclass labels (as defined in the original paper) for each cell in `cpm`.

▶ Click here for explanation how this was done

In [22]:
```
# Execute this block of code to assign cell subclass labels to all cells

# From the patchseq_metadata file downloaded from the Allen Institute websit
url = "https://raw.githubusercontent.com/nuoxuxu/gene-ephys-tutorial/main/da
metadata = pd.read_csv(urllib.request.urlopen(url), usecols = ["cell_specime
    .set_index("transcriptomics_sample_id")\
    .to_dict()["cell_specimen_id"]

# From the SpecimenMetadata file downloaded from the Allen Institute website
url = "https://raw.githubusercontent.com/nuoxuxu/gene-ephys-tutorial/main/da
ID_to_subclass = pd.read_csv(urllib.request.urlopen(url), usecols=["Specimer
    ['T type sub-class'].str.replace(' interneuron', '') \
    .to_dict()
```

In [23]:
```
# Convert column names from transcriptomics_sample_id to subclass labels
cpm = cpm.assign(subclass = cpm.index.map(metadata).map(ID_to_subclass))

# Subset for cells that belong to the 5 most abundant subclasses
cpm = cpm.loc[cpm.subclass.isin(["Pvalb", "Sst", "Sncg", "Lamp5", "Vip"])]

# Show the first 5 rows of the dataframe
cpm.head()
```

Out[23]:

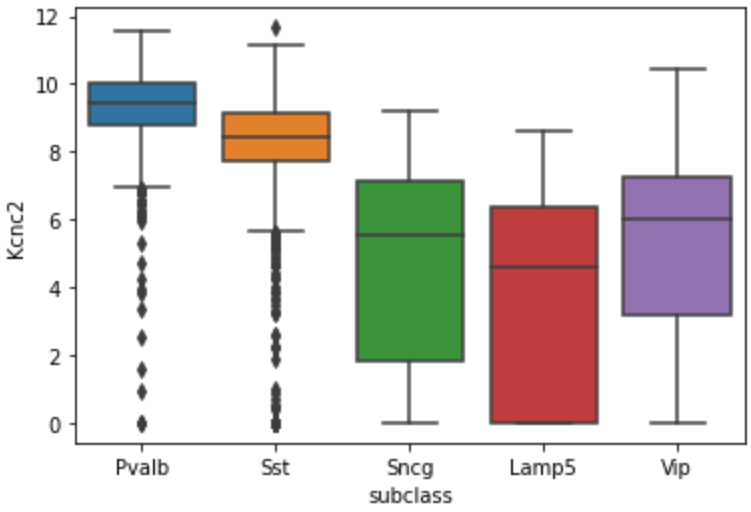
gene_name	0610005C13Rik	0610006L08Rik	0610007P14Rik	0610009B22Rik	0610
PS0810_E1-50_S88	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S19	0.0	0.0	0.000000	0.000000	
PS0817_E1-50_S25	0.0	0.0	0.000000	4.781173	
PS0817_E1-50_S26	0.0	0.0	5.876205	0.000000	
PS0817_E1-50_S27	0.0	0.0	6.084236	0.000000	

5 rows × 41931 columns

Expression of *Kcnc2* across subclasses

After assigning subclass label to each cell, we can plot the expression levels of genes across subclasses. Since *Kcnc2* is the most abundant gene in this dataset, we will plot the expression level of Kcnc2 across neuronal subclasses.

In [24]:
```
sns.boxplot(cpm[["subclass", "Kcnc2"]], x = "subclass", y = "Kcnc2",
            order = ["Pvalb", "Sst", "Sncg", "Lamp5", "Vip"])
plt.show()
```



This plot shows that *Kcnc2* expression is higher in the *Pvalb* interneurons, which are known to encompass fast-spiking interneurons.

UMAP visualization

Uniform manifold approximation and projection (UMAP) is a dimensionality reduction algorithm commonly used to visualize high-dimensional data on a two-dimensional space. Here, we can see that the cells that belong to the same subclass are close to each other in the 2D UMAP space.

Note: The cell below is computationally-intensive and takes several minutes to run. You can click on the box below to see the image it will produce.

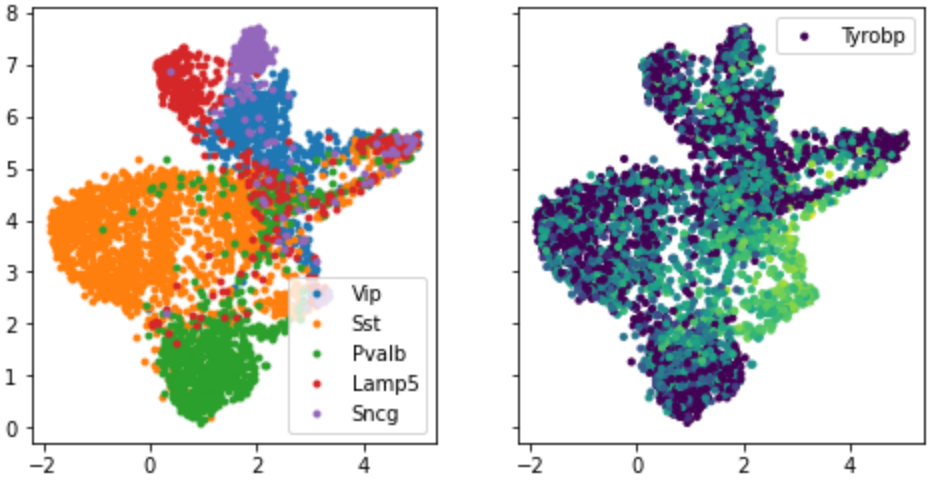
In [25]:
```
reducer = umap.UMAP()

# get the subclass and gene labels and sparse matrices from `cpm`
subclass_labels = cpm["subclass"]
gene_names = cpm.columns.values
sparse_cpm = sparse.csr_matrix(cpm.drop(columns = "subclass"))

embedding = reducer.fit_transform(sparse_cpm)
```

In [26]:
```
fig, axs = plt.subplots(1, 2, figsize = (8, 4), sharey = True)
for i, subclass in enumerate(subclass_labels.unique()):
    axs[0].plot(
        embedding[np.flatnonzero(subclass_labels == subclass), 0],
```

```
        embedding[np.flatnonzero(subclass_labels == subclass), 1],
        'o', markersize = 3,
        label = subclass)
axs[0].legend()
axs[1].scatter(embedding[:, 0], embedding[:, 1], s = 10, c = sparse_cpm[:, r
axs[1].legend()
plt.show()
```



▶ In case you do not get the cell above to run, here's what you should see!

Step 6. Plotting electrophysiological features across subclass

Importing extracted electrophysiological features

How do we know that *Pvalb* neurons are fast-spiking? We can import electrophysiological data from the `SpecimenMetadata` file

In [27]:
```
# You can import any ephys features that are contained in SpecimenMetadata
column_list = ['Specimen ID', 'Avg ISI', 'Fast trough V (long square) (milli
                   'Upstroke/downstroke ratio (long square)', 'Threshold V (

# import the ephys features from SpecimenMetadata, only using the columns in
ephys_data = pd.read_csv(
    urllib.request.urlopen("https://raw.githubusercontent.com/nuoxuxu/gene-e
    usecols=column_list, index_col=0)

# Assigning the subclass label to each cell
ephys_data["subclass"] = ephys_data.index.map(ID_to_subclass)

# Subset for cells that belong to the 5 most abundant subclasses
ephys_data = ephys_data.loc[ephys_data["subclass"].isin(["Pvalb", "Sst", "Sr
```

In [28]:
```
# This takes time to generate; it's not necessary to do so
ephys_data.head()
```

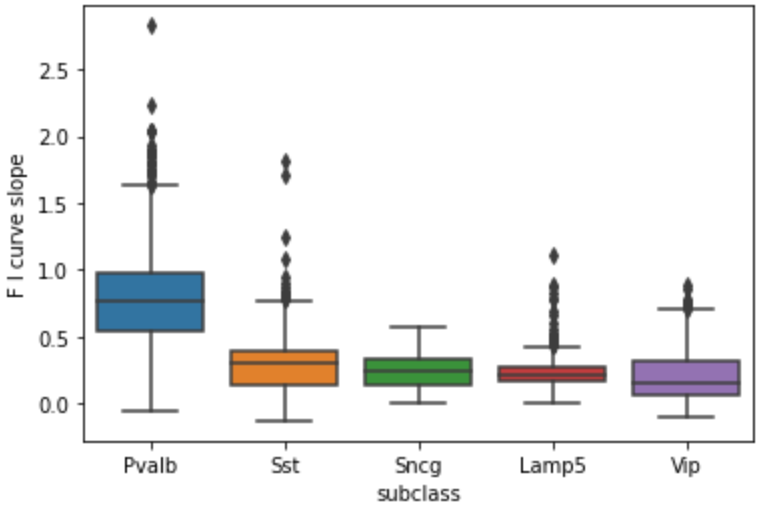
Out[28]:

Specimen ID	Avg ISI	Fast trough V (long square) (millivolts)	F I curve slope	Peak V (long square) (millivolts)	Peak V (short square) (millivolts)	sag	
888317699	77.337143	-49.899998	0.219149	33.968750	34.806252	0.088104	15.4
765853817	22.306667	-48.737499	0.700000	14.006250	14.231251	0.053379	41.7
671876425	87.811111	-52.343754	0.128299	15.562501	15.081251	0.010278	7.9
906699622	44.145000	-52.568752	0.075000	28.318752	28.806252	0.020501	22.5
778301240	9.276981	-49.618752	1.802804	27.556252	30.518749	0.086389	12.0

Plotting electrophysiological features across subclass

We can plot the values of any electrophysiological properties across the five subclass. *F I curve* is the slope of the curve between firing rate (f) and current injected. It is commonly used for distinguishing fast spiking interneurons. The figure shows that *Pvalb* neurons have higher *F I slope* than the rest.

In [29]:
```
sns.boxplot(ephys_data, x="subclass", y="F I curve slope", order=["Pvalb", "
plt.show()
```



Step 7. Correlating an electrophysiological feature with the expression of a gene

We have shown that *Kcnc2* is more abundant in Pvalb neurons and that Pvalb neurons have higher values of F I curve slope than other neuronal subclasses. looked at how gene expression levels and electrophysiological features could vary across neuronal subclasses, respectively. These two observations suggest that expression level of *Kcnc2* could be directly correlated with F I curve slope.

The unique advantage of Patch-seq is that we have both transcriptomic and electrophysiological data for the same cells, which allows us to directly correlate the two.

Plotting gene expression against electrophysiological feature

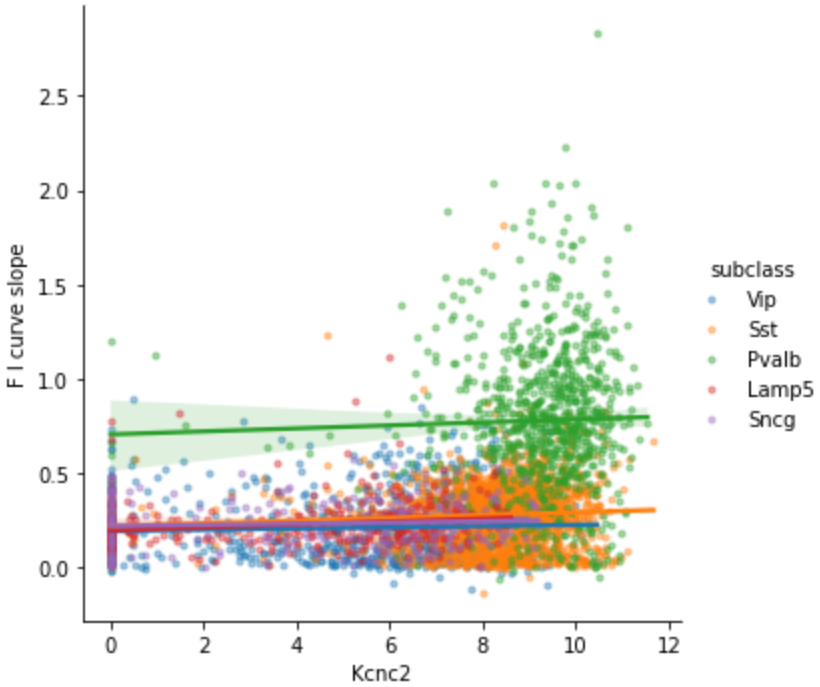
Here we are plotting the expression level of *Kcnc2* against F I curve slope, colored by subclass. You can substitute the electrophysiological features and genes with any entries in `cpm` and `ephys_data`.

In [30]:
```
F_I_slope = ephys_data[["F I curve slope", "subclass"]]
```

In [31]:
```
Kcnc2 = cpm["Kcnc2"]
Kcnc2.index = Kcnc2.index.map(metadata)
```

In [32]:
```
df = pd.merge(Kcnc2, F_I_slope, how = "inner", left_index = True, right_inde
```

In [33]:
```
sns.lmplot(df, x = "Kcnc2", y = "F I curve slope", hue = "subclass", scatter
plt.show()
```



Because of technical variability in scRNA-seq data, we need to pool raw read counts from cells that belong to the same transcriptomic type (one neuronal subclass contains multiple transcriptomic types), and then normalize and transform the pooled read counts. As a result of this operation, we will have one expression value for each gene for one transcriptomic type. Then we will correlated this "pooled" expression value with the median electrophysiological values of cells within the corresponding transcriptomic type.

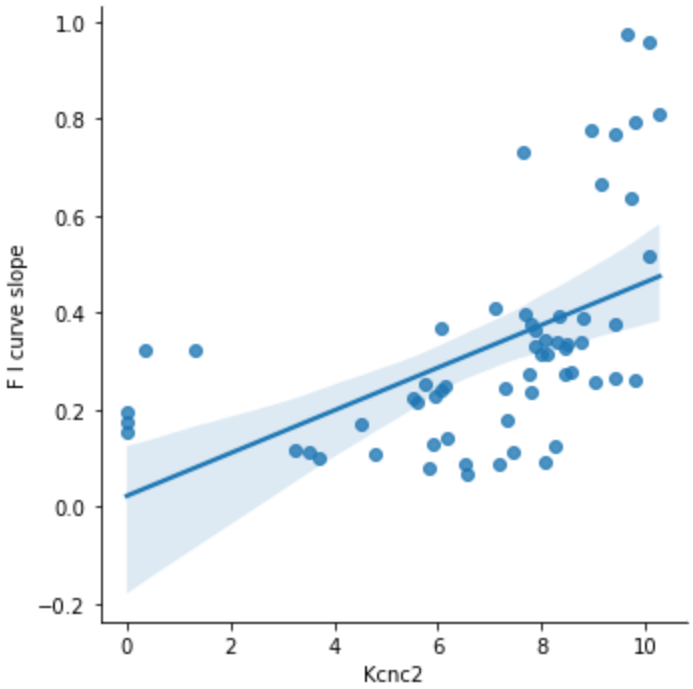
In [34]:
```
# Obtain mapping from transcriptomic_specimen_ID to T type
ID_to_ttype = pd.read_csv(urllib.request.urlopen(url), usecols=["Specimen ID
    ['T type']\
    .to_dict()
```

In [35]:
```
# Get median F I curve slop values for each T type
median_F_I_slope = ephys_data.assign(ttype = ephys_data.index.map(ID_to_ttyp
```

In [36]:
```
# Get median expression values of Kcnc2 for each T type
median_Kcnc2 = cpm.assign(ttype = cpm.index.map(metadata).map(ID_to_ttype)).
```

In [37]:
```
# Concatenate `median_F_I_slope` and `median_Kcnc2` into a Pandas DataFrame
Kcnc2_F_I_slope = pd.concat([median_Kcnc2, median_F_I_slope], axis = 1)
```

In [38]:
```
sns.lmplot(Kcnc2_F_I_slope, x = "Kcnc2", y = "F I curve slope")
plt.show()
```



This plot shows that there is a positive correlation between T type-specific expression of *Kcnc2* and *F I curve slope*.

Step 8. Constructing Gene-ephys Correlation Matrix

In the *Kcnc2* example we looked at how the expression of one gene could be correlated to one electrophysiological feature. We can compute correlations between all ion channel genes and all electrophysiological features in the dataset.

Subsetting for genes that code for voltage-gated ion channels

Below, the `IC_list` file contains a dictionary of genes that encode for ion channels.

In [39]:
```
# Download the list of voltage-gated ion channel genes (VGICs)
IC_list = json.load(urllib.request.urlopen("https://raw.githubusercontent.cc

# remove genes that are not in `cpm`
IC_list['VGIC'].remove("Trpc2")
IC_list['VGIC'].remove("Scn2a")
```

To construct a matrix where each entry is the Pearson's correlation coefficient between one electrophysiological property and expression level of one gene, we need to make sure that exactly the same cells are present in both DataFrames and that they are in the same order.

In [40]:
```
cpm_ttype = cpm.assign(ttype = cpm.index.map(metadata).map(ID_to_ttype)).drc
```

In [41]:
```
ephys_data_ttype = ephys_data.assign(ttype = ephys_data.index.map(ID_to_ttyp
```

Calculating the correlation matrix between the expression of ion channel genes and electrophysiology features

In [42]:
```
corr_matrix = np.corrcoef(
    np.hstack([cpm_ttype.to_numpy(), ephys_data_ttype.to_numpy()]), rowvar=F
        [:cpm_ttype.shape[1], -ephys_data_ttype.shape[1]:]
```

```
/opt/conda/lib/python3.9/site-packages/numpy/lib/function_base.py:2829: Runt
imeWarning: invalid value encountered in true_divide
  c /= stddev[:, None]
/opt/conda/lib/python3.9/site-packages/numpy/lib/function_base.py:2830: Runt
imeWarning: invalid value encountered in true_divide
  c /= stddev[None, :]
```

Plotting the correlation matrix

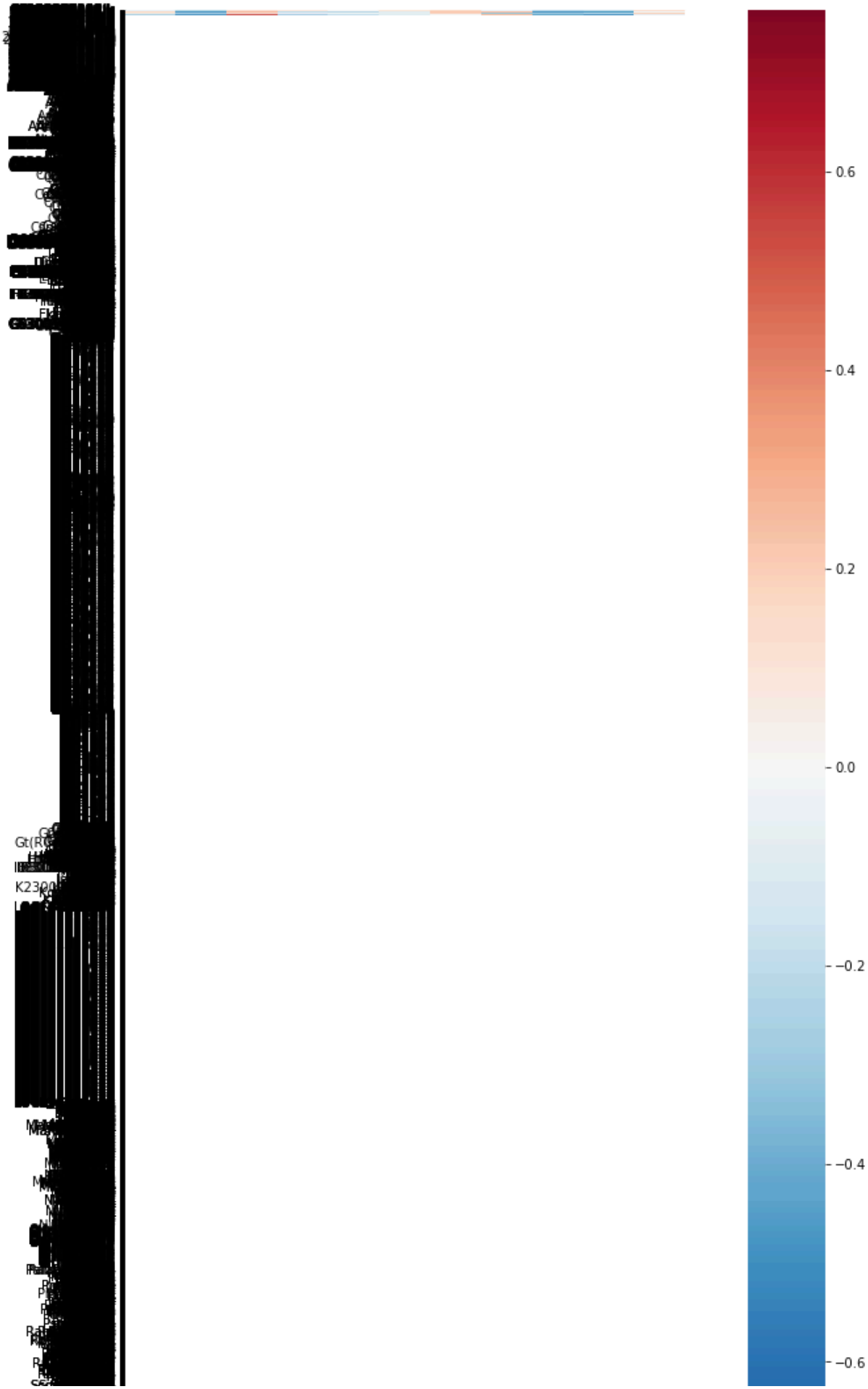
Note: This cell will take some time to generate!

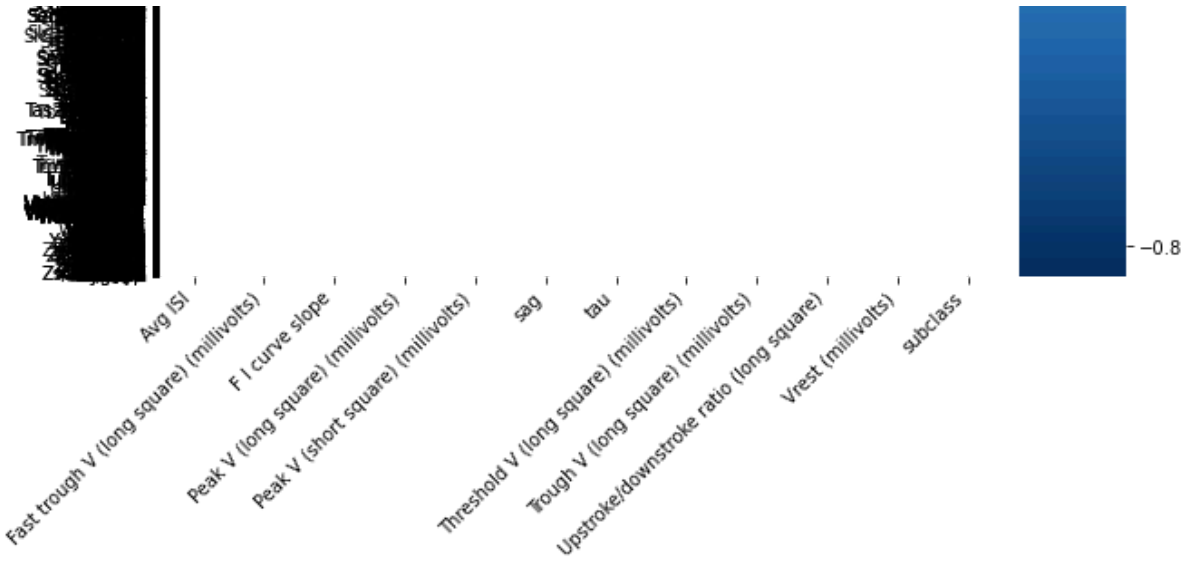
In [43]:

```
fig, ax = plt.subplots(figsize = (10, 22))
sns.heatmap(
    corr_matrix, cmap = "RdBu_r", center = 0,
    xticklabels = ephys_data.columns,
    yticklabels = cpm.columns
    )

# rotate the xticklabels
plt.xticks(rotation=45, ha='right')

plt.show()
```


0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6



▶ In case you do not get the cell above to run, here's what you should see!

About this notebook

This notebook was created by Nuo Xu, a physiology PhD student at the University of Toronto. It was edited for classes at UC San Diego by Ashley Juavinett.